Subject : Programming for Problem Solving Lab.

Unit 3 : Functions

Program : B.Tech. - CSE

Course Code : **CSCR 1501**

**Semester : 1st**

**Faculty Teaching : Dr. Shree Harsh Attri**

**Designation : Associate Professor**

**Department : Computer Sc. & Engineering**

Sharda University, Greater Noida, U.P

# Functions

- A **function** is a group of statements that together perform a specific task. Every C program has at least one function, which is **main()**
- Functions just need to be defined once in the program and then can be used multiple times.
- This also improves code readability and modularity while also providing code reusability.

**Functions are divided into two types:**

- **Library functions:** not need to write a code for a library function. It is already present inside the header file.
  Example: printf(), scanf(), getchar(), strlen(), strcpy() etc.

- **User-defined functions:** user has to write a body of a function and call the function whenever it requires.
  user-defined function in C is always written by the user, but later it can be a part of 'C' library.

# User defined Functions

User defined functions are divided into three activities:

1. **Function declaration:** means writing a name of a program. A function declaration is also called "Function **prototype**" and are usually done above the main() function.

   **Example**

   ```
   int add(int x, int y);
   ```

2. **Function definition:**

   A function definition provides the actual body of the function. It contains the block of code to perform a specific task. When a function is called, the control of the program is transferred to the function definition and the compiler starts executing the codes inside the body of a function.

   **Example**

   ```
   int add(int a, int b) //function body
   {
         int c;
         c = a + b;
         return c;
   }
   ```

3. **Function call:** A function call means calling a function whenever it is required in a program. It performs an operation for which it was designed.

   **Example**

   ```
   result = add(4,5);
   ```

# User defined Functions

```c
#include<stdio.h>
int add(int, int); // function prototype/declaration
main()
{
    int a, b, result;
    printf("Enter the values of a and b\n");
    scanf("%d %d",&a,&b);
    result = add(a,b);  // function calling
    printf("\n%d is the sum of %d and %d",result,a,b);
}
int add(x,y)    // function definition
{
    int sum;
    sum = x + y;
    return sum;
}
```

```
Enter the values of a and b
25
25

50 is the sum of 25 and 25
```

# User defined Functions

## Call by value

```c
#include<stdio.h>
int swap(int, int);     // function prototype/declaration
main()
{
    int a, b, result;
    printf("Enter the values of a and b\n");
    scanf("%d %d",&a,&b);
    printf("\nBefore swapping a = %d and b = %d",a,b);
    swap(a,b);          // function calling
}
int swap(x,y)           // function definition
{
    int temp;
    temp = x;
    x = y;
    y = temp;
    printf("\n\n After swapping a = %d and b = %d",x,y);
}
```

```
Enter the values of a and b
5
8

Before swapping a = 5 and b = 8

 After swapping a = 8 and b = 5
```

# Passing Arrays to Functions

## 1. Pass Individual Array Elements

```c
#include <stdio.h>
print(int,int,int,int,int);          //function protoype/declartion
main()
{
  int arr[5],i;
  printf("\nEnter the elements in an array of size 5:\n");
  for(i=0;i<5;i++)
  {
    scanf("%d", &arr[i]);
  }
  print(arr[0], arr[1], arr[2], arr[3], arr[4]);    //function calling
}
print(a,b,c,d,e)        //function definition
{
  printf("Value at 1st poisition is %d\n", a);
  printf("Value at 2nd poisition is %d\n", b);
  printf("Value at 3rd poisition is %d\n", c);
  printf("Value at 4th poisition is %d\n", d);
  printf("Value at 5th poisition is %d\n", e);
}
```

```
Enter the elements in an array of size 5:
9
8
7
6
5
Value at 1st poisition is 9
Value at 2nd poisition is 8
Value at 3rd poisition is 7
Value at 4th poisition is 6
Value at 5th poisition is 5
```

# Passing Arrays to Functions

## 2. Pass 1D Arrays to function

```c
#include <stdio.h>
int sum(int arr[]);          //function protoype/declartion
main()
{
  int arr[5],i, result;
  printf("\nEnter the elements in an array of size 5:\n");
  for(i=0;i<5;i++)
  {
    scanf("%d", &arr[i]);
  }
  result = sum(arr);     //function calling
  printf("Sum of the elements of an Array is: %d", result);
}
int sum(int arr[])       //function definition
{
  int add = 0, i;
  for(i=0;i<5;i++)
  {
    add = add + arr[i];
  }
  return add;
}
```

```
Enter the elements in an array of size 5:
1
2
3
4
5
Sum of the elements of an Array is: 15
```

# Call by Reference

**When function passes the addresses of a variables instead of values is called as call by reference.**

Function parameter **passes the address of original parameter** due to which they will point to same memory location and any changes made in the function parameter will also reflect in original parameters.

**Brief introduction of Pointer**

**Pointer is a variable which stores memory address of another variable as a value**

```
int x = 5;
int *ptr;     // pointer declaration
ptr = &x;
```

& □ address of memory
* □ pointer

# Call by Reference

SHARDA UNIVERSITY
Beyond Boundaries

## Understand the concept of Pointers

```c
#include<stdio.h>
main()
{
    int x = 5;
    int *ptr;
    ptr = &x;
    printf("Value of variable x is %d", x);
    printf("\nAddress of variable x is %d", &x);
    printf("\n\nValue of pointer variable ptr is %d",ptr);
    printf("\nAddress of pointer variable ptr is %d",&ptr);
    printf("\nValue at pointer variable ptr is %d",*ptr);
}
```

```
Value of variable x is 5
Address of variable x is 6422268

Value of pointer variable ptr is 6422268
Address of pointer variable ptr is 6422264
Value at pointer variable ptr is 5
```

# Call by Reference

Function parameter **passes the address of original parameter** due to which they will point to same memory location and any changes made in the function parameter will also reflect in original parameters.

```c
#include <stdio.h>
swap(int *,int *);      //function prototype
int main(){
    int x,y;
    printf("Enter the values of x and y:\n");
    scanf("%d %d",&x,&y);
    printf("\nValues before swap: x = %d, y = %d\n", x,y);
    swap(&x,&y);        //function calling
    printf("\nValues after swap: x = %d, y = %d\n", x,y);
}

swap(int *a, int *b)        //function definition
{
    int temp = *a;
    *a = *b;
    *b = temp;
}
```

// * is used to access the memory location that pointer is pointing

```
Enter the values of x and y:
5
6

Values before swap: x = 5, y = 6

Values after swap: x = 6, y = 5
```

# Exercise

Write the following Programs using Functions

1) To do the addition of two numbers and display the addition and maximum of two integers.
2) To calculate factorial of a given number
3) To print Fibonacci series up to specified range. (Consider the first term as 0 and second term as 1).
4) To take an array and its size as function argument and returns the average of all the elements.
5) Write a program using function to calculate the square and cube of a number.
6) Write a program using function to check whether a given number prime or not.
7) Write a program to demonstrate passing an array as an argument and perform the following using function:

   (a) To find largest number in an array
   (b) To print the array in reverse order

8) To sum two integers using

   (a) Call by value

   (b) Call by reference (using pointer)

# Pre-Processor Directives

# What is Pre-Processor?

Preprocessor is a program that processes the source program before it is passed to the complier.
Preprocessor commands often known as directives. These directives can be placed anywhere in a program but are most often placed at the beginning of a program, just before `main()`.

**Features of Preprocessor:**

The preprocessor offers several features called Preprocessor DirectivesPreprocessing directives are lines in a program that start with #.
The # is followed by an identifier that is the directive name.

The preprocessor directives are divided into following manners:

| Preprocessor Directives | Syntax/Description |
|---|---|
| Macro | Syntax: #define<br>This macro defines constant value and can be any of the basic data types. |
| File Inclusion | Syntax: #include <file_name><br>The source code of the file “file_name” is included in the main program at the specified place. |
| Conditional Compilation | Syntax: #ifdef, #ifndef, #if, #endif, #else, #elif<br>Set of commands are included or excluded in source program before compilation with respect to the condition. |
| Other Directives | Syntax: #undef, #pragma<br>#undef is used to undefine a defined macro variable. #pragma is used to call a function before and after main function in a C program. |

# What is Macro?

- Macro is a name which is given to a value or to a piece of code/block in a program.
- Instead of using the value, the macro is used which will replace the value in a program.
- A macro is defined with the preprocessor directive, **#define**.
- The reason for using macro is, consider below example:
  - ✔ *If a person's age is used as 50 in many places in a program and if direct value of the age i.e. 50 is used in all the places in program, it is very difficult to change the age value in future if it is required to change it to 60.*
  - ✔ *If macro is used in program, it is very simple to replace the value and it will change in only one place which will change the value in all places in a program.*

**Syntax:** `#define MACRO_NAME VALUE`

**Example:**

```
Original declaration – #define AGE 50
Modified declaration – #define AGE 60
```

# Ways to declare Macros

SHARDA UNIVERSITY
Beyond Boundaries

**a)**

```
#define UPPER 25    // macro declaration
main()
{
    int i;
    // use of macro in for loop
    for(i=1; i<=UPPER; i++)
    printf("\n %d",i);
}
```

**b)**

```
#define PI 3.14          // macro declaration
main()
{
    float r = 5.5;
    float area;
    // use of macro in value declaration
    area = PI * r * r;
}
```

**c)** **Macros used to define Operators**

```
#define AND &&      // macro declaration
#define OR ||   // macro declaration
main()
{
   int f = 1, x = 4, y = 90;
  // use of macro in conditional statement
   if ((f < 5) AND (x <= 20 OR y <= 45))
}
```

**d)** **Macros used to replace C statement**

```
#define PRINT printf("Hello");    //macro declaration
main()
{
    PRINT       // use of macro to replace statement
}
```

# Ways to declare Macros

## e) Macros with Arguments or Function – Like Macro

The function-like macro includes a list of parameters. They are known as function-like macros because they are used to stimulate functions. When a function is stimulated using a macro, the macro definition replaces the function definition.

**Example :**

```
#include<stdio.h>
#define SQUARE(x)  (x*x)        // macro declaration with arguments
main()
{
  int n,s;
  printf("Enter a number : ");
  scanf("%d", &n);
  s=SQUARE(n);              // perform action x * x
  printf("Square of given number = %d",s);
}
```

# # Operator

- ANSI C provides operator **#** called **stringizing** operator which is to be used only in function-like macros.
- This operator converts a formal argument into a string.

For example, if the macro is defined as follows:

**Example 1:**

```
#include<stdio.h>
#define PRINT(num)printf(#num " = %d",num)
main()
{
PRINT(20); // printf("20" " = %d",20);
}
```

**Output:**

20 = 20

**Example 2:**

```
#include <stdio.h>
#define PRINT(a, b)  printf(#a " and " #b ": Sweet Fruit!\n")
main()
{
PRINT(Apple,Mango);
}
```

**Output:**

Apple and Mango: Sweet Fruit !

# ## Operator

- ## is called **Merge** operator or **Token Pasting** operator.
- This concatenate two tokens into a third valid token.
- The token pasting (##) operator simply eliminates any white space around it and concatenates (joins together) the non-whitespace characters together.
- It can only be used in a macro definition.

```
#include<stdio.h>
#define CONCATENATE(a, b) a##b

int main() {
    int myVar = 100;

    // Concatenating variable name by macro
    printf("Value of myVar: %d\n", CONCATENATE(my, Var));

    // Another example: concatenating numbers
    int combinedNumber = CONCATENATE(123, 456);
    printf("Combined number: %d\n", combinedNumber);

    return 0;
}
```

# Exercise

Write the following Programs using Macros

1)To calculate the square of a number.

2)To calculate the cube of a number using another macro-SQUARE.

3)To compute the area of a circle using another macro for PI.

4)To calculate the maximum of five numbers.

5)To returns the absolute value of a number.